BITS PILANI • WORK INTEGRATED LEARNING PROGRAMMES

Professional Certificate in AI/ML • Module 05

LESSON 5

Kubernetes, Serverless & Deployment Patterns

From Container to Production — Scaling AI Workloads on Cloud, Edge & Hybrid

LEVEL Foundational ⟶ Intermediate | **DURATION** ~2.5 hours | **LAB** Real Amazon EKS deployment

Instructor-led session • BITS Pilani Off-Campus WILP

Learning Objectives

What you'll be able to do by the end of this lesson

01

Understand K8s architecture

Explain the control plane / worker node split and deploy AI containers on Kubernetes clusters

02

Use serverless for AI

Deploy lightweight inference with AWS Lambda, Cloud Functions, and the Truss framework

03

Choose deployment patterns

Apply decision criteria to pick between cloud, edge, and hybrid strategies for real workloads

Lesson Roadmap

Six topics building from K8s fundamentals to production deployment

1

Kubernetes Core Concepts

Architecture, pods, services, autoscaling

2

Deploying AI Models on K8s

Manifests, GPU, probes, storage, networking

3

Serverless AI Inference

Lambda, Cloud Functions, Truss, cold starts

4

Deployment Patterns

Cloud, edge, hybrid — how to choose

5

Hands-on — Amazon EKS

Real cluster: Minikube locally, EKS in cloud

6

Recap & Takeaways

Decision matrix, pattern summary, what next

Topic 1 — Kubernetes Architecture

Control plane, worker nodes, and pods — the building blocks

CONTROL PLANE — the brain that decides

API Server	Scheduler	Controller Mgr	etcd
Front-door for all requests	Decides where pods run	Reconciles desired ↔ actual	Cluster state store

WORKER NODES — where pods actually run

Node 1

kubelet · kube-proxy · container runtime

Pod	Pod	Pod
container	container	container
model.py	*model.py*	*model.py*

Node 2

kubelet · kube-proxy · container runtime

Pod	Pod	Pod
container	container	container
model.py	*model.py*	*model.py*

K8s Core Objects & Autoscaling

What you'll actually write in YAML

Pod	ReplicaSet	Deployment	Service
Smallest unit — 1+ containers	Keeps N replicas alive	Rolling updates + ReplicaSets	Stable DNS, load balance

Three Types of Autoscaling

HPA · Horizontal Pod

Add MORE pods

TRIGGER

CPU / memory / custom

VPA · Vertical Pod

Give pods MORE resources

TRIGGER

Historical usage

CA · Cluster

Add MORE nodes

TRIGGER

Unschedulable pods

Topic 2 — Deploying AI Models on Kubernetes

Manifests, GPU scheduling, health probes

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bert-inference
spec:
  replicas: 3
  selector:
    matchLabels: {app: bert}
  template:
    metadata:
      labels: {app: bert}
    spec:
      containers:
      - name: model
        image: ecr/bert:v1.2
        resources:
          limits:
            nvidia.com/gpu: 1
            memory: "8Gi"
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 30
```

GPU Scheduling

nvidia.com/gpu resource; requires NVIDIA device plugin

Readiness Probe

K8s won't route traffic until /health returns 200

Liveness Probe

Failed check ⟶ K8s restarts the container

Persistent Volumes

Model checkpoints, fine-tune data, artifacts

Rolling Updates

maxSurge and maxUnavailable control the strategy

Kubernetes Networking for AI Serving

Services and Ingress — how traffic reaches your model

Service Types

Type	Scope	Use Case	Cost
ClusterIP	Internal only	Service-to-service in cluster	Free
NodePort	Node IP + port	Dev / testing exposure	Free
LoadBalancer	Public cloud LB	Production external endpoint	~$18/mo (AWS)
Ingress	L7 HTTP routing	One LB → many services by path	~$18/mo total

Typical AI Serving Traffic Path

Ingress (NGINX/ALB) → Service (ClusterIP) → Pods (multiple replicas) → Inference

Topic 3 — Serverless AI Inference

Lambda, Cloud Functions, and Truss — when to skip K8s

K8s vs Serverless

Dimension	K8s	Serverless
Scale to zero	No	Yes
Cold start	None	1–30s
Max runtime	Unlimited	15 min
Model size	Any	≤ 10 GB
Cost — low QPS	Higher	Lower
Cost — high QPS	Lower	Higher
GPU support	Yes	Limited

What is Truss?

Open-source Python framework by Baseten.

Standard directory — model.py + config.yaml + requirements.txt

One-command deploy — Baseten, AWS, GCP, or K8s

Handles boilerplate — HTTP server, health checks

Cold-Start Problem

Mitigation strategies:

- Provisioned concurrency (Lambda)
- Model quantization (smaller = faster)
- Container image caching
- Warm-up ping schedules
- SnapStart for JVM runtimes

Topic 4 — Deployment Patterns

Cloud, edge, and hybrid — three flavors, one decision

CLOUD

WHEN TO USE

Standard SaaS APIs, batch processing

✓ PROS

- Elastic scaling
- Managed infra
- Rich services

✗ CONS

- Data leaves premises
- Network latency

EDGE

WHEN TO USE

IoT, mobile, real-time <50ms, privacy

✓ PROS

- Ultra-low latency
- Data stays local
- Offline capable

✗ CONS

- Resource-limited
- Hard fleet updates

HYBRID

WHEN TO USE

Regulated data, mixed workloads

✓ PROS

- Best of both
- Compliance-friendly
- Failover ready

✗ CONS

- Higher complexity
- Two ops stacks

Choosing a Deployment Pattern

Decision matrix — pick by latency budget and data sensitivity

Criterion	Cloud (K8s)	Serverless	Edge	Hybrid
Latency budget	< 500 ms	< 2s (cold: 30s)	< 50 ms	Mixed
Throughput	High, steady	Spiky low-med	Low, local	Segmented
Data sensitivity	Depends	Depends	High	Very high
Model size	Any	≤ 10 GB	≤ 500 MB	Split by tier
Cost @ low QPS	$$	$	$$$	$$$
Cost @ high QPS	$	$$$	N/A	$$
Ops complexity	Medium	Low	High	Very high

RULE OF THUMB — start with the simplest pattern that meets your latency + compliance requirements. Add complexity only when metrics prove you need it.

Topic 5 — Hands-on: Minikube + EKS

Same YAML, two targets — local dev, cloud deploy

LOCAL — Minikube

Full K8s cluster in a VM on your laptop

```
minikube start
```
Boot cluster (~2 min)

```
kubectl get nodes
```
Verify node Ready

```
kubectl apply -f dep.yaml
```
Deploy your manifest

```
minikube service my-svc
```
Open in browser

```
minikube dashboard
```
Troubleshooting UI

CLOUD — Amazon EKS

Managed control plane, you bring the nodes

```
eksctl create cluster
```
VPC + IAM + control plane

```
aws eks update-kubeconfig
```
Point kubectl at EKS

```
kubectl get nodes
```
Confirm workers joined

```
kubectl apply -f dep.yaml
```
SAME manifest as Minikube

```
eksctl delete cluster
```
Full teardown & cleanup

Common Pitfalls & How to Avoid Them

Real-world mistakes that break production AI deployments

01 **No readiness probe**

Traffic hits pod before model loads → 502s. Always set /health with initialDelaySeconds ≥ load time.

02 **OOMKilled**

Container hits memory limit → K8s kills it → crash loop. Set limits by peak batch size, not average.

03 **Secrets in ConfigMaps**

ConfigMaps are readable by anyone with pod access. Use Secrets — with KMS encryption at rest.

04 **Forgotten LoadBalancer**

One LB per Service = $18/mo each. Use Ingress to consolidate services behind one LB.

05 **Cold-start neglect**

Serverless works in dev, dies at scale if you don't plan for 10–30s cold starts on large models.

06 **No resource limits**

One rogue pod hogs node memory → others die → cascade failure. Always set requests + limits.

Key Takeaways

What you can now design and deploy

01 — K8s architecture

Control plane decides, workers run — the split matters

02 — Core objects

Deployment → ReplicaSet → Pod, exposed by Service

03 — Three autoscalers

HPA, VPA, CA — often used together

04 — AI-specific manifests

GPU limits, readiness probes tuned for load time

05 — Serverless is a choice

Great for spiky low-QPS; watch cold starts and 15-min cap

06 — Truss abstracts deploy

One config, one command, multiple targets

07 — Pattern decision matrix

Latency + data sensitivity drive cloud/edge/hybrid

08 — Same YAML everywhere

Minikube → EKS with just a kubeconfig switch

Thank You

Questions & Discussion

What's Next

Complete the Lab

Deploy the notebook to a real EKS cluster end-to-end

GitOps with ArgoCD

From kubectl apply → git-driven declarative deploys

KServe / Kubeflow

Purpose-built ML serving frameworks on K8s

Cost discipline

Always eksctl delete cluster after every lab session